

图像处理基础课程课后作业十一

建智 2211 谢天赐 21221182144

11.1.1)

代码:

```
import cv2
import numpy as np

# 读取图片
image = cv2.imread('11-3.jpg')

# 获取图片的高度、宽度和通道数
height, width, channels = image.shape

# 存储白色点坐标的列表
white_points_coordinates = []

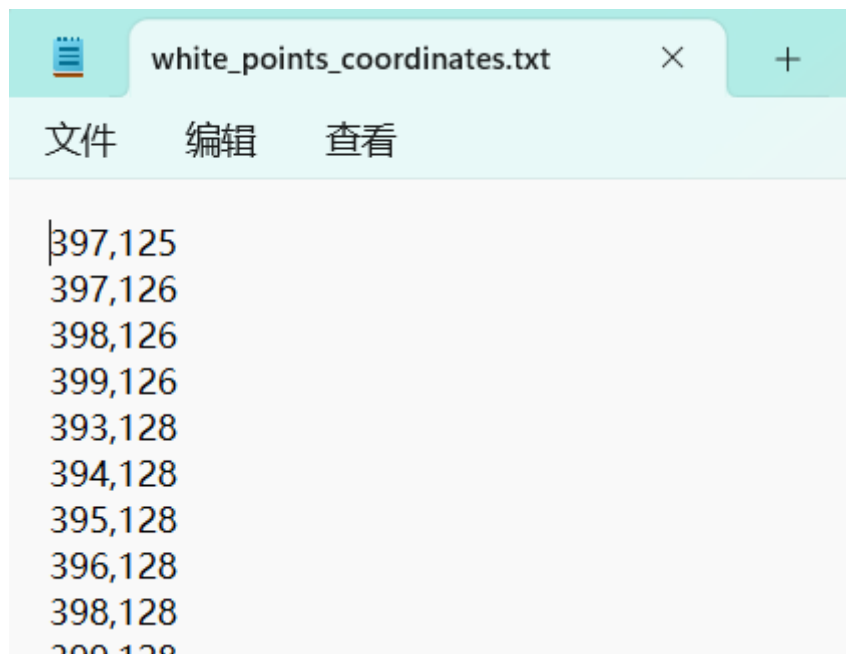
# 遍历图片的每个像素
for y in range(height):
    for x in range(width):
        # 获取当前像素的RGB值
        b, g, r = image[y, x]

        # 判断是否为白色点, 这里设定RGB值都大于等于240视为白色点, 可根据实际情况调整
        if r >= 240 and g >= 240 and b >= 240:
            white_points_coordinates.append((x, y))

# 打印白色点的坐标
for point in white_points_coordinates:
    print("白色点坐标:", point)

# 将白色点坐标保存为文本文件
with open('white_points.txt', 'w') as f:
    for point in white_points_coordinates:
        f.write(f"{point[0]},{point[1]}\n")
```

程序运行结果如图所示:



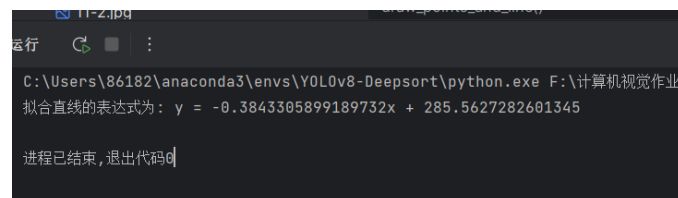
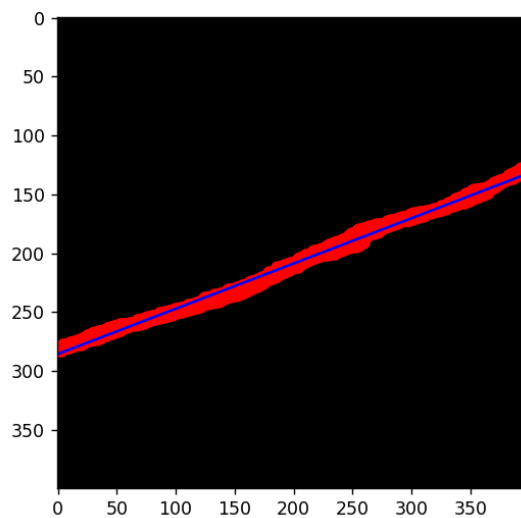
直线坐标文本



圆弧坐标文本

11.1.2)

求解直线参数:



主要代码:

```

show.py  nihe.py x  circle.py  2.py  3.py  11-2.jpg

def read_points_from_text(file_path):
    points = []
    with open(file_path, 'r') as f:
        lines = f.readlines()
        for line in lines:
            x, y = line.strip().split(',')
            points.append([float(x), float(y)])
    return np.array(points)

def fit_straight_line(points):
    X = points[:, 0].reshape(-1, 1)
    y = points[:, 1]
    model = LinearRegression()
    model.fit(X, y)
    return model.coef_[0], model.intercept_

def draw_points_and_line(image_path, points, slope, intercept):
    image = cv2.imread(image_path)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

    plt.scatter(points[:, 0], points[:, 1], c='r', marker='o')

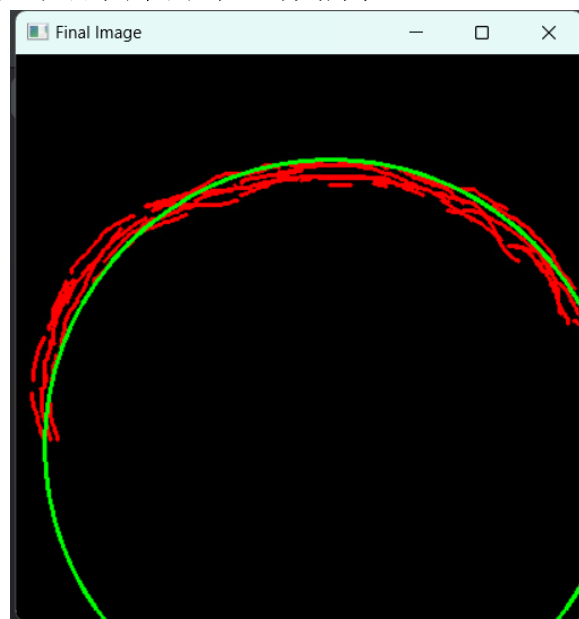
    x_min, x_max = np.min(points[:, 0]), np.max(points[:, 0])
    x_fit = np.array([x_min, x_max])
    y_fit = slope * x_fit + intercept

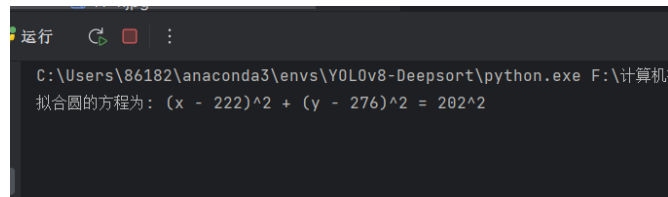
    plt.plot(x_fit, y_fit, c='b')

```

11.1.3)

求解圆弧参数，在原图中的对比与其方程：





```
运行 C:\Users\86182\anaconda3\envs\YOL0v8-Deepsort\python.exe F:\计算机
拟合圆的方程为: (x - 222)^2 + (y - 276)^2 = 202^2
```

主要代码:

```
import cv2
import numpy as np
# 读取文本文件中的点坐标数据
def read_points_from_text(file_path):
    points = []
    with open(file_path, 'r') as f:
        lines = f.readlines()
        for line in lines:
            x, y = line.strip().split(',')
            points.append([float(x), float(y)])
    return np.array(points)

# 将点绘制到图片上
def draw_points_on_image(points, image_path):
    image = cv2.imread(image_path)
    for point in points:
        int_point = (int(point[0]), int(point[1]))
        cv2.circle(image, int_point, 1, (0, 0, 255), -1)
    return image

# 执行霍夫圆检测并绘制拟合圆
def fit_circle_with_hough(points, image_path):
    image = draw_points_on_image(points, image_path)
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blurred_image = cv2.GaussianBlur(gray_image, (5, 5), 0)
    edges = cv2.Canny(blurred_image, 50, 150)

    circles = cv2.HoughCircles(edges, cv2.HOUGH_GRADIENT, 1, 150,
                                param1=70, param2=30, minRadius=0, maxRadius=0)

    if circles is not None:
        circles = np.uint16(np.around(circles))
        for i in circles[0, :]:
            cv2.circle(image, (i[0], i[1]), i[2], (0, 255, 0), 2)

    # 获取圆心坐标和半径
```

```

        x0 = i[0]
        y0 = i[1]
        r = i[2]

        # 输出拟合圆的方程
        print(f"拟合圆的方程为: (x - {x0})^2 + (y - {y0})^2 = {r}^2")

    return image

if __name__ == "__main__":
    file_path = 'white_points_circle.txt'
    image_path = '11-2.jpg'

    points = read_points_from_text(file_path)

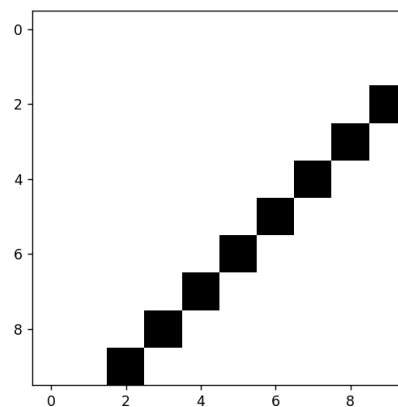
    final_image = fit_circle_with_hough(points, image_path)

    cv2.imshow("Final Image", final_image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

```

11.2

1) 绘制的表格点:



2) 确定霍夫直线变换参数:

```

# 2. 确定霍夫直线变换的参数
max_distance = 15  max_distance: 15
angle_range = np.deg2rad(np.arange(-90, 90))  angle_range: [-1.57079633 -1.55334303 -1.53588974 -1.51843645 -1.50098316 -1.48352986
distance_accuracy = 1  distance_accuracy: 1
angle_accuracy = np.deg2rad(1)  angle_accuracy: 8.017453292519943295

```

3) 判断行列数等:

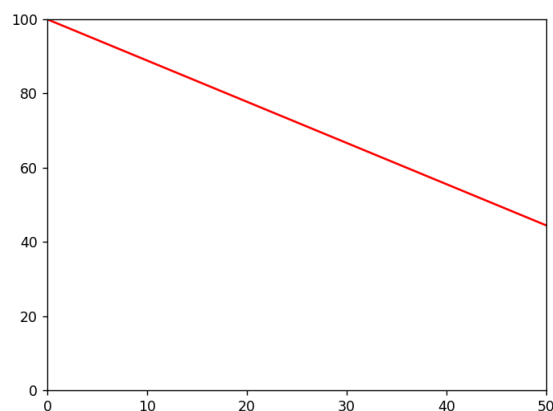
```
# 3. 计算参数rho, theta对应的可能函数点数值及计数器表格行列数
rho_values = np.arange(0, max_distance + 1) rho_values: [0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15]
theta_values = angle_range theta_values: [-1.57079633 -1.55354383 -1.53588974 -1.51843645 -1.50098316 -1.48352986, -1.46607657 -1.44862328 -1.43116999 -1.41371669 -1.3962
counter_table = np.zeros((len(rho_values), len(theta_values))) counter_table: [[0. 0. 0. ... 0. 0. 0.], [0. 0. 0. ... 0. 0. 0.], [0. 0. 0. ... 1. 1. 1.], ..., [0. 0. 0. ... 0. 0. 0.]]
```

4、5) 计数器表格统计并检测到霍夫直线参数:

	127	128	129	130	131	132	133	134	135	136	137	138	139	140
0	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
1	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
2	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
3	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
4	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
5	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
6	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
7	0.0000	6.0000	6.0000	6.0000	6.0000	7.0000	8.0000	8.0000	8.0000	8.0000	8.0000	7.0000	6.0000	6.0000
8	0.0000	2.0000	2.0000	2.0000	2.0000	1.0000	0.0000	0.0000	0.0000	0.0000	0.0000	1.0000	2.0000	2.0000
9	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
10	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
11	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
12	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
13	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
14	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
15	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

```
#
file_path = 'white_points2.txt' file_path: 'white_points2.txt'
image_points = read_points_from_txt(file_path) image_points: [(9, 2), (8, 3), (7, 4), (6, 5), (5,
# 执行操作
draw_table_and_fill(image_points)
counter_table = hough_transform(image_points)
rho, theta = find_hough_line_parameters(counter_table) rho: 7 theta: 0.7504915783575618
draw_detected_line(rho, theta)
```

6) 绘制直线:



代码实现:

```
import numpy as np
import matplotlib.pyplot as plt
# 从 txt 文件读取坐标点数据
def read_points_from_txt(file_path):
    points = []
    with open(file_path, 'r') as file:
        for line in file.readlines():
            x, y = map(int, line.strip().split(','))
            points.append((x, y))
```

```

    return points

# 1. 绘制 10x10 表格并涂黑相应点
def draw_table_and_fill(image_points):
    table = np.zeros((10, 10)) + 255
    for point in image_points:
        x, y = point
        table[y, x] = 0
    plt.imshow(table, cmap='gray')
    plt.show()

# 2. 确定霍夫直线变换的参数
max_distance = 15
angle_range = np.deg2rad(np.arange(-90, 90))
distance_accuracy = 1
angle_accuracy = np.deg2rad(1)

# 3. 计算参数 rho、theta 相应的可能离散点数值及计数器表格行列数
rho_values = np.arange(0, max_distance + 1)
theta_values = angle_range
counter_table = np.zeros((len(rho_values), len(theta_values)))

# 4. 逐个判定白色点是否属于某条霍夫直线并填写计数器表格
def hough_transform(image_points):
    for x, y in image_points:
        for theta_idx, theta in enumerate(theta_values):
            rho = int(x * np.cos(theta) + y * np.sin(theta))
            if 0 <= rho < len(rho_values):
                counter_table[rho, theta_idx] += 1
    print(counter_table)
    return counter_table

# 5. 指出检测到的霍夫直线参数(rho, theta)
def find_hough_line_parameters(counter_table):
    rho_idx, theta_idx = np.unravel_index(np.argmax(counter_table),
counter_table.shape)
    print(rho_values[rho_idx], theta_values[theta_idx])
    return rho_values[rho_idx], theta_values[theta_idx]

# 6. 计算直线端点坐标并绘制直线（放大 10 倍显示）
def draw_detected_line(rho, theta):
    x1, y1 = 0, int(rho / np.sin(theta))
    x2, y2 = int(rho / np.cos(theta)), 0
    plt.plot([x1 * 10, x2 * 10], [y1 * 10, y2 * 10], 'r')

```

```

plt.xlim(0, 50)
plt.ylim(0, 100)
plt.show()

#
file_path = 'white_points2.txt'
image_points = read_points_from_txt(file_path)

# 执行操作
draw_table_and_fill(image_points)
counter_table = hough_transform(image_points)
rho, theta = find_hough_line_parameters(counter_table)
draw_detected_line(rho, theta)

```

11.3



```

C:\Users\86182\anaconda3\envs\Y0L0v8-De
矩数据表:
|  Order  | Moment Value |
|-----|-----|
|    0    | 544025.0     |
|  1 (x)  | 101721913.0  |
|  1 (y)  | 70373296.0   |
|  2 (xx) | 27339952665.0|
|  2 (yy) | 10349520112.0|
|  2 (xy) | 11795703187.0|

进程已结束,退出代码0

```

代码实现:

```

import cv2
import numpy as np

# 读取图像（这里假设图像是二值图像，你可以根据实际情况调整读取方式）
image_path = '11-2.jpg'
image = cv2.imread(image_path, 0)

# 计算图像的矩
moments = cv2.moments(image)

```



```

# 提取不同阶数的矩
order_0_moment = moments['m00']
order_1_moment_x = moments['m10']
order_1_moment_y = moments['m01']
order_2_moment_xx = moments['m20']
order_2_moment_yy = moments['m02']
order_2_moment_xy = moments['m11']

# 创建矩数据表
moment_data = {
    'Order': ['0', '1 (x)', '1 (y)', '2 (xx)', '2 (yy)', '2 (xy)'],
    'Moment Value': [order_0_moment, order_1_moment_x,
order_1_moment_y,
order_2_moment_xx, order_2_moment_yy,
order_2_moment_xy]
}

# 打印矩数据表
print("矩数据表: ")
print("|{: ^10}|{: ^15}|".format("Order", "Moment Value"))
print("|{: ^10}|{: ^15}|".format("-" * 10, "-" * 15))
for order, value in zip(moment_data['Order'], moment_data['Moment Value']):
    print("|{: ^10}|{: ^15}|".format(order, value))

```